

Date & Time Functions, Database, MySQL, Query, Prepared Statement

- Md. Morshedul Arefin

Date and Time Functions

✓ Current Date and Time:

```
$dt = date("Y-m-d H:i:s");  
echo $dt; // 2023-08-09 13:49:58
```

✓ Custom Date Formatting:

```
$ts = strtotime("2023-08-09 15:30:00");  
$formatted = date("F j, Y, g:i a", $ts);  
echo $formatted; // August 9, 2023, 3:30 pm
```

✓ Get different parts of a date:

```
$ts = time();  
$year = date("Y", $ts);  
$month = date("m", $ts);  
$day = date("d", $ts);  
echo "Year: $year, Month: $month, Day: $day"; // Year: 2023, Month: 08, Day: 09
```

Date and Time Functions (Continue ...)

✓ Calculate future or past dates:

```
$today = date("Y-m-d");  
$daysToAdd = 7;  
$futureDate = date("Y-m-d", strtotime($today . " + $daysToAdd days"));  
echo "Today: {$today} and after 7 days: {$futureDate}"; // Today: 2023-08-09 and  
after 7 days: 2023-08-16
```

✓ Timezone Handling:

```
$timezone = new DateTimeZone("America/New_York");  
$dt = new DateTime("now", $timezone);  
$formattedTime = $dt->format("Y-m-d H:i:s");  
echo "Time in New York: " . $formattedTime; // Time in New York: 2023-08-09  
09:56:57
```

Date and Time Functions (Continue ...)

✓ Time Difference:

```
$startTime = strtotime("2023-08-09 10:00:00");  
$endTime = strtotime("2023-08-09 15:30:00");  
$timeDiff = $endTime - $startTime;  
echo "Time Difference: " . gmdate("H:i:s", $timeDiff); // Time Difference: 05:30:00
```

✓ Get Day of Week:

```
echo "Today is ".date("l"); // Today is Wednesday
```

✓ Getting the Difference Between Two Dates:

```
$startDate = strtotime("2023-08-01");  
$endDate = strtotime("2023-08-15");  
$daysDifference = ($endDate - $startDate) / (60 * 60 * 24);  
echo "$daysDifference days"; // 14 days
```

Date and Time Functions (Continue ...)

✓ Getting the Difference in Years, Months, and Days:

```
$startDate = new DateTime("1984-09-25");  
$endDate = new DateTime("2023-08-09");  
$interval = $startDate->diff($endDate);  
echo "Difference: {$interval->y} years, {$interval->m} months, {$interval->d} days";  
// Difference: 38 years, 10 months, 15 days
```

```
$startDate = new DateTime("1984-09-25");  
$endDate = new DateTime("now");  
$interval = $startDate->diff($endDate);  
echo "Difference: {$interval->y} years, {$interval->m} months, {$interval->d} days";  
// Difference: 38 years, 10 months, 15 days
```

Date and Time Functions (Continue ...)

✓ Finding the Next Occurrence of a Specific Day of the Week:

```
$targetDay = "Wednesday";  
$nextWednesday = strtotime("next $targetDay");  
echo "Next $targetDay: " . date("Y-m-d", $nextWednesday); // Next Wednesday:  
2023-08-16
```

✓ Getting the Last Day of a Month:

```
$lastDayOfMonth = date("Y-m-t", strtotime("2023-08-09"));  
echo $lastDayOfMonth; // 2023-08-31
```

Date and Time Functions (Continue ...)

✓ Checking if a Date Falls within a Range:

```
$startDate = strtotime("2023-08-01");  
$endDate = strtotime("2023-08-15");  
$checkDate = strtotime("2023-08-10");  
if ($checkDate >= $startDate && $checkDate <= $endDate) {  
    echo "The date falls within the range.";  
} else {  
    echo "The date is outside the range.";  
}
```

✓ Formatting Relative Time (e.g., "2 hours ago"):

```
$timestamp = strtotime("2 hours ago");  
$relativeTime = date("F j, Y, g:i a", $timestamp);  
echo "Relative Time: $relativeTime"; // Relative Time: August 9, 2023, 12:06 pm
```

Date and Time Functions (Continue ...)

✓ Getting how many hours or times ago:

```
$currentTime = time();  
$postTime = strtotime("2023-08-02 15:30:00");  
$timeDifferenceInSeconds = $currentTime - $postTime;  
if ($timeDifferenceInSeconds < 60) {  
    $timeAgo = "just now";  
} elseif ($timeDifferenceInSeconds < 3600) {  
    $timeAgo = floor($timeDifferenceInSeconds / 60) . " minutes ago";  
} elseif ($timeDifferenceInSeconds < 86400) {  
    $timeAgo = floor($timeDifferenceInSeconds / 3600) . " hours ago";  
} elseif ($timeDifferenceInSeconds < 2592000) {  
    $timeAgo = floor($timeDifferenceInSeconds / 86400) . " days ago";  
} elseif ($timeDifferenceInSeconds < 31536000) {  
    $months = floor($timeDifferenceInSeconds / 2592000);  
    $timeAgo = "$months " . ($months > 1 ? 'months' : 'month') . " ago";  
} else {  
    $years = floor($timeDifferenceInSeconds / 31536000);  
    $timeAgo = "$years " . ($years > 1 ? 'years' : 'year') . " ago";  
}  
echo "Posted $timeAgo";
```


Database

- A Database is an organized collection of data stored and accessed electronically from a computer system.
- Instead of saving information in unstructured text files or spreadsheets, a database allows us to store, manage, update, and retrieve data efficiently and securely.
- In web development, databases are used to store dynamic content such as user credentials, blog posts, product inventories, and order histories.

DBMS (Database Management System)

- Software that interacts with end users, applications, and the database itself to capture and analyze data.

Table:

The fundamental structure of a relational database, consisting of rows (records) and columns (attributes).

MySQL

- MySQL is an open-source Relational Database Management System (RDBMS) that uses SQL (Structured Query Language) to manage data.
- In a relational database, data is stored in separate tables that can be linked or "related" to one another using primary and foreign keys.
- MySQL is fast, reliable, widely used with PHP, and powers millions of web applications worldwide.
- PHP sends SQL commands to MySQL, receives the results, and displays them on a web page.

Difference between DBMS and RDBMS

Features	DBMS (Database Management System)	RDBMS (Relational DBMS)
Data Storage	Stores data as files or in hierarchical / network structures.	Stores data in structured tables with rows and columns.
Relationships	Does not natively support relationships between datasets.	Links tables together using Primary Keys and Foreign Keys.
Normalization	Not supported (leads to redundant/duplicate data).	Supported (reduces data redundancy and saves space).
Data Volume	Best for small amounts of simple data.	Designed for large amounts of complex, connected data.
Security	Low security	High security
Examples	XML, File systems, MS Access (basic mode).	MySQL, PostgreSQL, Oracle, SQLite, SQL Server.

What is a Query?

- A Query is a request or command sent to a database to perform a specific action on stored data. It is written in SQL (Structured Query Language).
- Instead of opening database files manually, you send queries to retrieve specific records, insert new entries, modify existing details, or delete unwanted data.

Data Retrieval:

Requesting specific information from tables.

Data Modification:

Adding, updating, or deleting records.

Data Definition:

Creating, altering, or removing database structures (tables, columns).

Example

```
SELECT name, marks  
FROM students  
WHERE course = 'Web Development';
```

Prepared Statement?

- A Prepared Statement is a feature used to execute the same (or similar) SQL statements repeatedly with high efficiency and security.
- Instead of sending raw values directly inside a query, you use placeholders (?) or named parameters like :email).
- The database compiles the SQL query structure first, and then the actual parameters are sent separately. This completely separates SQL code from user input, making it the primary defense against SQL Injection attacks.

Insecure Query (Vulnerable to SQL Injection):

```
$email = $_POST['email'];
```

```
$sql = "SELECT * FROM users WHERE email = '$email'";
```


Secure Query (Prepared Statement with PDO):

Method 1:

Using Positional Placeholders (?)

Method 2:

Using Named Placeholders (:name) & Explicit Binding (bindParam)

Method 1:

Using Positional Placeholders (?)

// 1. Prepare statement with '?' placeholders

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = ? AND status = ?");
```

// 2. Execute by passing values in an array matching the placeholder order

```
$stmt->execute([$emailInput, $statusInput]);
```

```
$user = $stmt->fetch();
```

Method 2:

Using Named Placeholders (:name) & Explicit Binding (bindParam)

// 1. Prepare statement with named parameters

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email AND  
status = :status");
```

// 2. Bind parameters explicitly

```
$stmt->bindParam(':email', $emailInput, PDO::PARAM_STR);  
$stmt->bindParam(':status', $statusInput, PDO::PARAM_STR);
```

// 3. Execute statement

```
$stmt->execute();  
$user = $stmt->fetch();
```